

Nome: Mell Campos, Brayan Martins, Nicolas Marin

1. DATASET

Dataset 1: NYC Rat Sightings, disponível em <https://www.kaggle.com/datasets/new-york-city/nyc-rat-sightings>, que reúne registros de reclamações e avistamentos de roedores na cidade de Nova York com atributos que permitem análises geoespaciais e temporais, como data e hora, localização geográfica precisa, divisões administrativas e tipo de local do avistamento.

Dataset 2: DOHMH inspeções sanitárias/controle de pragas, mantido pelo Department of Health and Mental Hygiene da cidade de Nova York, que registra vistorias com data, localização, resultado (aprovado/reprovado) e tipo de evidência encontrada (ex.: tocas ativas), usado para construir a densidade de atividade de inspeção próxima a cada chamado. Disponível em: <https://data.cityofnewyork.us/Health/Rodent-Inspection/p937-wjvj>

Dataset 3: NOAA Global Historical Climatology Network - Daily (GHCN-Daily), estação Central Park, NY (USW00094728), disponível em <https://www.ncei.noaa.gov/data/global-historical-climatology-network-daily>, que fornece temperatura mínima, máxima e precipitação diárias, usadas para construir o sinal climático (com médias móveis de 3/7 dias e anomalia de temperatura).

Origem: Os dados do primeiro dataset são registros feitos por cidadão no portal 311 do Governo da Cidade de Nova York (requisições de serviços públicos), complementados por dados do Department of Health and Mental Hygiene (DOHMH), responsável pelas inspeções de saúde pública e controle de pragas na cidade. Os dados do segundo dataset são vistorias realizadas pelo próprio Department of Health and Mental Hygiene (DOHMH), responsável pelas inspeções de saúde pública e controle de pragas na cidade. Os dados do terceiro dataset foram coletados e mantidos pela NOAA (National Oceanic and Atmospheric Administration) na rede GHCN-Daily.

Adequação: O dataset de avistamentos permite análise preditivo e otimização de vistorias sanitárias, pois a quantidade de dados de cunho espacial e temporal permite mapear hotspots de infestação e investigar padrões sazonais, e o histórico de chamados reais mostra a pressão urbana. O dataset de inspeções DOHMH complementa essa leitura ao trazer uma medida direta da atenção institucional já dedicada a cada área, permitindo comparar densidade de fiscalização com gravidade confirmada de infestação. O dataset climático da NOAA traz um complemento no aspecto do clima, já que infestações e aparições de roedores estão ligadas ao calor e estações do ano, e cruzar a data do chamado com temperatura e precipitações permite observar como esse efeito se manifesta.

2. PROBLEMA E OBJETIVO

Problema analisado: A proliferação de ratos em centros urbanos densos como Nova York representa um desafio crítico de saúde pública, saneamento e qualidade de vida. Embora a prefeitura receba muitas denúncias, a distribuição espacial e temporal é muito heterogênea, e a ausência de uma leitura integrada sobre quais bairros tem maior incidência de avistamentos, quais estações intensificam as ocorrências e como a densidade populacional se relaciona a esses chamados impede a distribuição de inspeções sanitárias preditivas e eficientes. O tempo de resolução dos chamados também é uma variável determinante na contenção de doenças e na eficiência das operações da prefeitura.

Objetivo geral: Desenvolver um modelo preditivo capaz de classificar a resolução de chamados de avistamento em “Rápida” ou “Lenta”, apoiando a priorização de vistorias pelo DOHMH.

Objetivos específicos: Mapear a densidade de avvistamentos por Borough e ZIP Code pra identificar hotspots de infestação; Investigar a sazonalidade dos chamados, cruzando a frequência de denúncias com variáveis climáticas e temporais; Correlacionar avvistamentos com fatores contextuais; Classificar a eficiência de atendimento das denúncias a partir de um modelo supervisionado.

Tipo de tarefa de ML: Classificação binária supervisionada.

Fundamentação teórica da escolha do target: Em vez de tentar adivinhar o dia em que um chamado será resolvido, o modelo divide os dados entre os mais rápidos e os mais lentos a partir da mediana do tempo de resolução no conjunto de treino (~9 dias), o que cria grupos aproximadamente equilibrados. Isso evita que o sistema viciie, e entrega um alerta de risco de atraso, o que é mais prático para o dia a dia do que tentar prever a data exata.

3. EXPLORAÇÃO E PREPARAÇÃO DE DADOS

O dataset bruto de avvistamentos contém 102 mil registros entre 2010 e 2017, com colunas reduzidas a 15 atributos finais (Unique Key, Created Date, Closed Date, Location Type, Incident Zip, Status, Borough, Latitude, Longitude, year, month, day_of_week, hour, season, resolution_days). Na etapa exploratória, identificou-se um problema de qualidade de dado relevante: 16.840 chamados com Status “Pending” (ainda em aberto) apresentavam Closed Date corrompida no dado bruto, gerando tempos de resolução negativos (até -42 mil dias). Foi corrigido o problema usando Status == “Closed” e removendo outliers de tempo de resolução fora do intervalo de 0 a 365 dias antes de definir a variável alvo.

Mantemos os chamados com Status “Pending” ou “Assigned” no dataset, mesmo sem entrarem como exemplo de treino: eles são descartados na etapa de modelagem, mas seguem sendo contabilizados no cálculo da carga de trabalho simultânea, pois um chamado aberto e nunca fechado efetivamente ocupa a fila de atendimento. Se esses registros fossem descartados no início do pipeline, a feature de carga de trabalho não mostraria a congestão real do sistema.

Também foram construídas features de calendário com codificação cíclica (month_sin, month_cos, doy_sin, doy_cos) que representam meses e dias do ano em um círculo trigonométrico. Dessa forma, por exemplo, dezembro e janeiro ficam numericamente próximos, pois uma codificação inteira simples não conseguiria mostrar isso.

4. ENGENHARIA DE ATRIBUTOS

A engenharia de atributos foi a etapa que mais impactou o desempenho do modelo, com quatro grupos principais de features, tomando cuidado para usar as informações disponíveis antes da abertura de cada chamado.

- **Carga de trabalho simultânea (simultaneous_workload):** conta quantos chamados do mesmo Borough estavam ativos quando um chamado específico foi criado, como proxy de sobrecarga operacional. É calculada com um algoritmo de varredura (sweep-line): cada chamado gera dois eventos, abertura (+1) e fechamento (-1), limitado a uma janela de 90 dias pra evitar que um chamado específico de 2010 infle a fila pra sempre. Sem esse teto, a correlação da feature com o ano de denúncia chegava a 0,70. Com o teto, caiu pra 0,33, preservando o sinal verdadeiro de congestão em vez de um proxy falso.
- **Eficiência histórica da vizinhança (KNN):** pra cada chamado, se calcula a média do tempo de resolução dos 30 chamados geograficamente mais próximos que já haviam sido fechados antes do início do mês em questão, usando uma BallTree com métrica haversine (distância sobre a esfera terrestre). O valor k=30 foi escolhido após testes

entre 5 e 300 vizinhos: valores maiores elevavam o AUC até 0,7555 mas representavam ~35% de todo o histórico do bairro, deixando de captar uma vizinhança local de fato. $k=30$ preserva o sentido conceitual da feature.

- **Sinal climático (NOAA):** temperatura máxima/mínima e precipitação da estação Central Park, com médias móveis de 3 e 7 dias. Isoladamente, o clima do dia teve pouco efeito; a média móvel captou melhor um efeito acumulado (o comportamento dos roedores não reage no instante em que a temperatura sobe).
- **Densidade das inspeções DOHMH:** contagem de inspeções sanitárias num raio de 300 metros, nos 365 dias anteriores ao mês do chamado, também via BallTree/haversine. O achado mais relevante desse bloco (testado por raio, endereço exato e tipo de evidência específica como “tocas ativas”) foi que a densidade de atenção da prefeitura na área prevê o tempo de resolução, enquanto a gravidade confirmada da infestação (falhas de inspeção, multas, etc) teve importância próxima a zero nos três testes.

Na versão mais recente do modelo (V10), esse conjunto foi expandido com seis features adicionais, cuja mais relevante isoladamente foi `vel_recente_borough_30d_90d` (tempo médio de fechamento no borough nos 30/90 dias anteriores ao mês), que mede o “regime de atendimento vigente” (vê como está agora, e não como é num geral). Outras adições incluem Address Type e Community Board como variáveis categóricas, a hora da abertura do chamado, e o histórico causal por endereço (número de chamados anteriores no mesmo local e dias desde o último, usando `cumcount` sobre ordenação temporal).

5. BLINDAGEM CONTRA DATA LEAKAGE

- **No cálculo do alvo:** a mediana usada para binarizar o tempo de resolução em “Rápida” / “Lenta” é calculada exclusivamente sobre o conjunto de treino, nunca sobre o dataset completo, o que evita que o limiar de corte carregue informação do conjunto de teste.
- **Nas features de vizinhança e inspeção:** o pool de candidatos usado no KNN inclui apenas chamados com Closed Date anterior ao início do mês avaliado, nunca Created Date, que permitiria a um vizinho ter fechado depois do chamado atual e vazear informações do futuro. Esse é geralmente o ponto mais frágil de features desse tipo.
- **No ColumnTransformer do pipeline:** colunas como Closed Date, Status e Unique Key permanecem na matriz de features X até o pré processamento, mas o parâmetro `remainder = 'drop'` garante que nunca cheguem ao classificador. Funciona, mas o parâmetro deve permanecer o mesmo pra continuar funcionando no futuro.
- **Na codificação de categorias:** o TargetEncoder do scikit-learn realiza cross-fitting interno, o valor codificado de cada linha é calculado com dobras que não incluem essa própria linha, evitando que uma codificação simples por média do alvo vazee informações diretamente.
- **Um bug de integridade de dado foi identificado e corrigido:** a função de cálculo da carga de trabalho simultânea retornava um objeto `pandas.Index` em vez de uma `Series` com índice alinhado. Se atribuíssemos esse retorno de volta a uma coluna do `DataFrame`, o `pandas` não conseguiria alinhar os rótulos e atribuía os valores por posição, sem gerar erro ou aviso. A correção foi substituir `.map().fillna(0)` por `pd.Series(resultado).reindex(df.index).fillna(0)`, e melhorou o desempenho no modelo. Isso é uma evidência de que o problema era ruído por desalinhamento, e não vazamento de dados.

6. ESTRATÉGIA EXPERIMENTAL

O conjunto de dados final (72.091 registros, após filtragem de chamados fechados válidos) foi dividido em treino e teste na proporção 80/20, com seed fixa (42) para reprodutibilidade. A busca de hiperparâmetros foi feita com RandomizedSearchCV, realizando 25 combinações aleatórias avaliadas em validação cruzada de 5 dobras, otimizando a métrica AUC sem, em nenhum momento, ter contato com o conjunto de teste.

Além da avaliação padrão com split aleatório, foi conduzido um segundo experimento com split temporal: o modelo foi treinado com todos os dados até agosto de 2016 e testado apenas no período seguinte, simulando o cenário real de uso (prever chamados que ainda vão acontecer, não chamados sorteados do mesmo período de treino). Essa comparação foi central para avaliar a real capacidade de generalização do modelo, e não apenas seu desempenho sob uma divisão artificial dos dados.

7. MODELAGEM

Na etapa exploratória, sete algoritmos foram comparados (KNN, Árvore de Decisão, SVM com kernel RBF, Random Forest, Gradient Boosting, Bagging e Regressão Logística), com os ensembles de árvores apresentando o melhor desempenho (acurácia entre 0,69 e 0,70). A partir dessa comparação, optou-se por abandonar Random Forest e ensembles combinados, consolidando o HistGradientBoostingClassifier (scikit-learn) como modelo final, escolhido também por aceitar Latitude/Longitude crus como entrada, aprendendo fronteiras de decisão por cortes sucessivos, sem necessidade de um agrupamento geográfico prévio (como KMeans).

O pipeline final combina um ColumnTransformer com três ramos: StandardScaler para variáveis numéricas (sem efeito relevante para modelos baseados em árvore, mas inofensivo), TargetEncoder para variáveis categóricas de alta cardinalidade (Borough, Location Type, Address Type, Community Board) e passagem direta (passthrough) para variáveis já numéricas, como latitude/longitude e as features de engenharia.

Na versão V10, a busca de hiperparâmetros revelou que o modelo estava subtreinado: o grid original limitava max_iter a 200 sem early_stopping, insuficiente para ~57 mil linhas de treino com cerca de 30 features. Ampliando o grid para max_iter entre 300 e 400, com early_stopping=True (que interrompe o treino automaticamente caso o desempenho pare de melhorar, sem risco de overfitting adicional) e permitindo até 63 folhas por árvore (max_leaf_nodes=63), obtiveram-se os parâmetros finais: learning_rate=0.1, max_iter=400, max_depth=None, max_leaf_nodes=63, min_samples_leaf=50, l2_regularization=1.0, early_stopping=True. Esse ajuste isolado foi responsável pelo maior ganho de desempenho do projeto (~+0,03 de AUC), superando a soma de todas as features novas adicionadas na mesma versão.

8. AVALIAÇÃO

Ao longo do projeto, o modelo passou por evolução de desempenho. Na versão inicial (V1), um Random Forest foi treinado sobre os dados brutos e comparado com outros seis algoritmos, alcançou AUC de 0,600. A introdução da feature de carga de trabalho simultânea, da troca para o HistGradientBoosting e do uso direto de latitude/longitude (V7) elevou o AUC para 0,698. A adição do sinal climático da NOAA, com médias móveis de 3 e 7 dias, levou o AUC a 0,720. Em seguida, a inclusão da eficiência histórica de vizinhança via KNN causal elevou o resultado para 0,740, e a densidade de inspeções DOHMH em raio de 300

metros levou o AUC a 0,752. A versão consolidada dessa primeira fase (V9) fechou em 0,744 de AUC de teste.

Na versão mais recente (V10), a correção do bug de desalinhamento na feature de carga de trabalho, somada a seis novas features e ao ajuste dos hiperparâmetros do HistGradientBoosting, elevou o AUC de teste para 0,7836, o melhor resultado do projeto.

Junto com esse ganho, a acurácia no limiar ótimo chegou a 0,7078, e a precisão nos 20% de chamados sinalizados como mais lentos (a métrica mais relevante para o uso prático do modelo como ferramenta de triagem) subiu de 75,0% no V9 para 84,5% no V10.

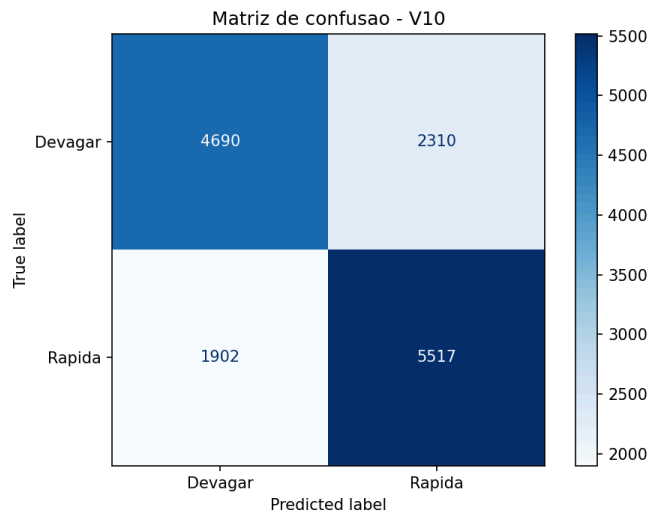
A importância por permutação no conjunto de teste do modelo final, que mede a queda de AUC ao embaralhar cada coluna, e é considerada mais confiável que a importância de impureza, apontou como preditores mais relevantes, em ordem: dias_desde_2010 (0,0407), Address Type (0,0293), vel_recente_borough_30d (0,0271), simultaneous_workload (0,0211), Latitude (0,0163), doy_sin (0,0145), Longitude (0,0120), Community Board (0,0120), vel_recente_borough_90d (0,0115) e local_viz_mediana (0,0100).

O achado mais relevante desta etapa, no entanto, veio da avaliação com split temporal, treinando o modelo com todos os dados até agosto de 2016 e testando apenas no período seguinte, em vez do split aleatório usado até então. Enquanto o split aleatório indicou AUC entre 0,73 e 0,78, o split temporal revelou uma queda acentuada para a faixa de 0,58 a 0,64, resultado praticamente idêntico entre V9 e V10. Isso descarta a hipótese de que as features novas seriam responsáveis pela diferença: a causa é estrutural, já que o regime de atendimento da prefeitura muda ano a ano, e o split aleatório acaba misturando chamados do mesmo período (por vezes do mesmo dia) nos dois lados da divisão, entregando ao modelo uma vantagem que não existiria em uma aplicação real de previsão do futuro.

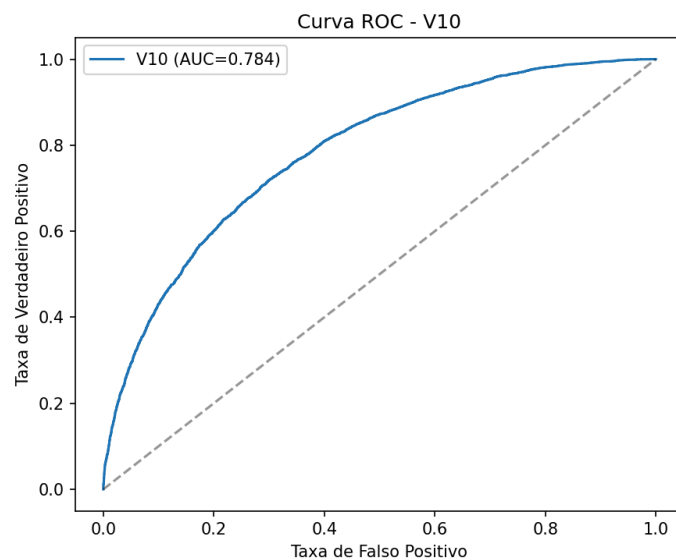
Versão	O que mudou	AUC teste
V1 (baseline)	Random Forest, dados brutos, 7 algoritmos comparados	0,600
V7	Carga de trabalho simultânea + HistGradientBoosting + lat/long cru	0,698
+ clima	Temperatura/chuva (NOAA) com médias móveis 3/7 dias	0,720
+ vizinhança	Eficiência histórica KNN causal	0,740
+ inspeção DOHMH	Densidade de inspeções em raio de 300m	0,752
V9 (consolidado)	Pipeline final da primeira fase	0,744

V10	Correção do bug + 6 features novas + hiperparâmetros ajustados	0,7836
-----	--	--------

Matriz de confusão:



Curva de ROC:



9. ANÁLISE DE RESULTADOS

Comparando o desempenho em validação cruzada (AUC de 0,7748, medido durante a busca de hiperparâmetros) com o AUC obtido no conjunto de teste (0,7836), não há indício de overfitting: os valores são próximos, e o de teste é até levemente superior, o que reforça a generalização do modelo dentro do período de dados observado (ressalva feita à parte na discussão de split temporal).

Analisando o comportamento do modelo por faixa de confiança, aproximadamente 28% dos chamados caem em uma região onde o modelo está confiante (probabilidade prevista abaixo de 30% ou acima de 70%), com acerto de ~79% nesses casos. Por outro lado, cerca de 20% dos chamados caem em uma "zona cega" (probabilidade entre 45% e 55%), onde a acurácia (~51%) é estatisticamente equivalente a um chute aleatório, resultado esperado, e não uma falha do modelo, já que reflete casos genuinamente ambíguos nos dados (dois chamados resolvidos em 8 e 10 dias, por exemplo, são na prática quase equivalentes frente a uma mediana de ~9 dias).

Esse padrão reforça que o uso mais defensável do modelo é como ferramenta de triagem/priorização, não como um oráculo de decisão individual: ao selecionar os 20% de chamados com maior probabilidade prevista de serem "lentos", entre 75% e 84% deles de fato o são (contra 50% esperado ao acaso), e esse grupo leva, em média, significativamente mais dias para ser resolvido que a média geral.

Dentre as tentativas testadas e descartadas, destacam-se: a combinação Borough × Location Type no cálculo de carga de trabalho, que piorou o desempenho (-0,038) por gerar combinações esparsas (quase metade com menos de 100 chamados); o histórico de inspeção por endereço exato, que se mostrou neutro ou pior, já que apenas a densidade de atenção importa, não a gravidade; e a evidência específica de inspeção ("tocas ativas"), que não trouxe ganho perceptível (+0,0004), pela mesma razão. Um alvo alternativo também foi testado, isolando apenas os extremos (25% mais rápidos vs. 25% mais lentos, descartando o meio), que elevou o AUC para 0,80-0,81, mas às custas de descartar 48% dos dados, não foi adotado como resultado principal, servindo antes como evidência de que o teto de desempenho em ~0,74-0,78 reflete uma ambiguidade real dos dados próximos da mediana, e não uma limitação do modelo.

A leitura honesta do projeto, portanto, é dupla: sob a premissa de que o modelo seria periodicamente re-treinado com dados recentes (Framing 1), o desempenho do split aleatório é uma avaliação razoável de uma ferramenta de priorização operacional de curto prazo. Já sob uma leitura mais rigorosa de generalização (Framing 2), a validação temporal expõe que boa parte do sinal aprendido reflete "em que época/regime este chamado está", informação que um split aleatório entrega de graça e que não estaria disponível na prática ao prever chamados futuros de verdade.

10. DEMONSTRAÇÃO DE FUNCIONAMENTO

[Link do vídeo.](#)

(Se houver algum problema com o vídeo pode entrar em contato que envio direto).